A Network-Aware Transport for KV-Cache Communication Between LLM Agents

Soham Gupta, Dhruv Patel, Yash Ravipati

Overview

- Multi-agent LLMs traditionally communicate through text, requiring recomputation of internal representations.
- Newer approaches share latent representations or KV caches, potentially reducing redundant computation.
    - However, KV-cache transfers can involve hundreds of MB per message, and their efficiency is unclear.
- **Our goal:**
    - **Determine when latent communication (KV cache transfers) outperforms text across real networks**
    - **Build a runtime layer that adaptively selects the communication format**

What problem are we solving?

Text communication minimizes network traffic but requires the receiver to recompute internal states. KV-cache communication avoids redundant computation but introduces substantial network transfer costs.

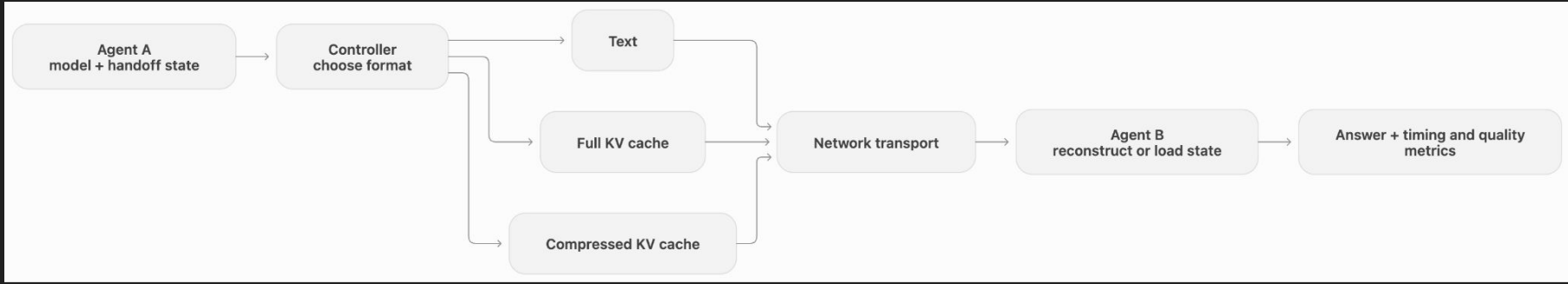
- Existing studies primarily focus on inference efficiency and task accuracy, leaving performance in distributed network systems relatively unclear.
    - **LatentMAS** shows that passing latent state between agents can improve inference speed: **4.3× faster on average** than text baseline. However, it does not establish whether these speedups persist when agents communicate across machines, where network transfer overhead may outweigh the computation saved.

Our question: ***Distributed across several machines, when does each format give the best end-to-end latency at an acceptable accuracy?***

Why is it an important problem?

- Distributed multi-agent systems increasingly span multiple GPUs and machines for scalability and specialization.
  - Transferring hundreds of MB per message could eliminate the speedups achieved by avoiding redundant computation.
  - Different network conditions, context lengths, and receiver workloads change which communication format performs best.
- Fixed communication strategies may introduce unnecessary network traffic or computation.
  - **A network-aware runtime layer could adapt communication decisions to improve end-to-end performance.**

Technical architecture involved



- **Jev** is a fast decision model that returns structured choices and probabilities rather than generated text. typesafe.ai
- In this project, Jev could help the controller choose **text, compressed KV, or full KV** at each agent handoff by estimating how much fidelity that handoff needs.
- We would compare it with a simpler controller based on bandwidth, transfer size, and receiver compute. Jev is useful only if its choices improve latency or accuracy **after counting the time spent calling it**.

What challenges might we face?

a. **Balancing performance and accuracy:** Compressing KV caches can reduce communication costs but may affect downstream answer quality. We need to measure this tradeoff across different tasks and compression levels.
b. **Making effective communication decisions:** The best choice between sending tokens, full KV caches, or compressed caches depends on bandwidth, context length, and receiver load. Our controller must adapt without introducing excessive overhead that negates the purpose of our approach.
c. **Demonstrating improvements:** We must compare against strong baselines and identify when adaptive communication provides meaningful benefits over a fixed strategy.

Midterm Goals

Prototype [Soham]: Build and deploy a two-agent prototype across separate machines that supports text, full KV-cache, and compressed KV-cache communication.

Controller [Yash]: Build a controller that selects the communication format for each message based on network conditions and estimated computation costs.

Evaluation [Dhruv]: Set up and run experiments comparing communication strategies across different network conditions, measuring latency, communication volume, and task accuracy.

Appendix: Background Work

- **LatentMAS** shows that passing latent state between agents can improve inference speed: **4.3× faster on average** than its hierarchical text baseline. Its result does not establish when transferring that state across machines is worthwhile.
- **KV compression work** reduces the size of agent handoffs. One study reports **290 MB for full KV versus about 62 MB for compressed KV**, but evaluates a single-process chain where the cache incurs no network transfer time.
- **Adaptive transport work** has modeled when tokens or KV caches should be sent. Its multi-agent evaluation is simulation based; CacheGen demonstrates bandwidth-adaptive KV or text transfer for serving reusable context.

On real machines, when does text, full KV, or compressed KV give the best end-to-end latency at an acceptable task accuracy?

Appendix: Related Work

- **Dai et al. (2026)**: Model when agents should send text or KV caches, but evaluate the decision through simulation rather than measured cross-machine handoffs.
- **CacheGen (2024)**: Adapts compressed KV transfer to network bandwidth, with text recomputation as a fallback, but focuses on reusable serving context rather than agent communication.
- **Li et al. (2026)**: Measure cache size, compression cost, and answer quality for latent agent relay, but run agents in one process without network transfer.

Our project: Compare text, full KV, and compressed KV across machines, measuring end-to-end latency and task accuracy; test whether an adaptive controller beats fixed formats.

Q&A